

This section installs the tools you'll need, gives you a quick taste of each layer, and provides the mental model for the rest of the lab.

1.1 Install uv

[uv](#) is a fast Python package manager from Astral. Kiro uses `uvx` (included with `uv`) to run MCP servers without manual installation.

Step 1: Install `uv`:

macOS / LinuxWindows (PowerShell)

```
1 curl -L$SF https://astral.sh/uv/uv/install.sh | sh
2
3 # Or via Homebrew
4 brew install uv
```

Step 2: Verify the installation:

```
1 uv --version
2 uvx --help
```

Why uvx?

`uvx` lets you run Python-based MCP servers on the fly without installing them globally. Many Kiro Powers rely on it, so having `uv` installed is a prerequisite for using MCP-powered features.

Ref: [uv Installation Guide](#)

1.2 Kiro Directory Structure: Global vs Workspace Scope

Kiro uses two directory scopes to organize steering files, skills, and agents. Understanding this structure is important before you start creating them.

Scope	Definition	Priority
Global	Stored in your home directory (<code>~/.kiro/</code>). Applies across ALL workspaces on your machine. Ideal for personal preferences, team-wide conventions pushed via MDM, or rules you want everywhere.	Lower - overridden by workspace
Workspace	Stored in your project root (<code>./.kiro/</code>). Applies only to that specific project. Ideal for project-specific standards, deployment procedures, or team conventions for that repo.	Higher - wins over global

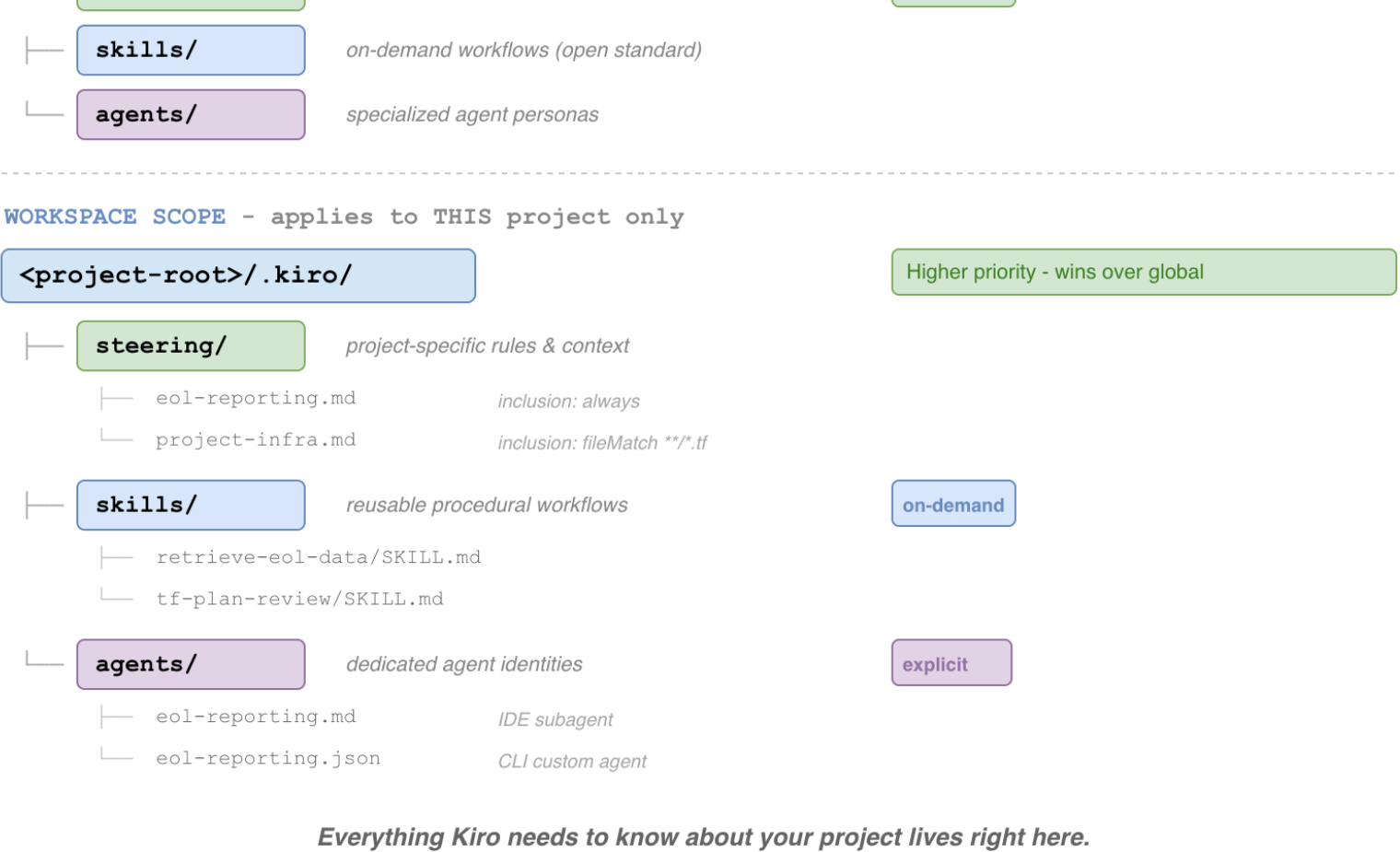
What is ~ (home directory)?

~ is a shorthand for your user's home directory.

- macOS: `/Users/<username>/`
- Linux: `/home/<username>/`
- Windows: `C:\Users<username>\` (accessible via `$env:USERPROFILE` in PowerShell)

For example, `~/.kiro/steering/` means `/Users/jane/.kiro/steering/` on macOS.

Directory layout:



Example:

Type	Global Path	Workspace Path	On Conflict
Steering	<code>~/.kiro/steering/</code>	<code>.kiro/steering/</code>	Workspace wins
Skills	<code>~/.kiro/skills/</code>	<code>.kiro/skills/</code>	Workspace wins (by name)
Agents	<code>~/.kiro/agents/</code>	<code>.kiro/agents/</code>	Both available

Note

When both global and workspace versions exist with the same name, the workspace version takes priority. This lets you set global defaults and override them per project as needed.

Ref: [Steering Docs](#) | [Skills Docs](#) | [Agents Docs](#)

1.3 Install Global-scoped Steering Files

Global steering files live in `~/.kiro/steering/` and apply across all your workspaces. They are ideal for personal coding standards, team-wide conventions, or behavioral rules you want Kiro to follow everywhere.

Step 1: Create the global steering directory:

macOS / LinuxWindows (PowerShell)

```
1 mkdir -p ~/.kiro/steering
```

Step 2: Open `~/.kiro/steering/generic-rules.md` in Kiro IDE (run `kiro ~/.kiro/steering/generic-rules.md`, click "Yes" if a pop-up appears) and paste the following:

```
1 ---
2 inclusion: always
3 ---
4
5 > ** MANDATORY **: Every instruction in this file is non-negotiable. Follow all rules exactly - no exceptions, no skipping, no partial compliance.
6
7 # Behavior
8
9 - Don't hold back. Give it your all.
10 - Skip flattery and respond directly to questions.
11 - Don't use your own knowledge to answer questions - always verify with authoritative sources.
12 - When the ask is related to AWS, always check AWS documentation first.
13 - After receiving tool results, reflect on their quality and plan next steps before proceeding.
14
15 # Efficiency
16
17 - Run multiple tools simultaneously whenever operations are independent.
18 - If you create any temporary files, scripts, or helpers, clean them up when the task is done.
19 - Never run multi-line commands directly. Instead, write a helper script to a temp directory, execute it, then delete it.
20
21 # Formatting
22
23 - Use ` ` instead of ` ` (en dash).
24 - Do not use emoji in responses unless the user explicitly asks for them.
25
26 # Code Quality
27
28 - Write general-purpose solutions that work for all valid inputs - never hard-code for specific test cases.
29 - Implement the actual algorithm, not a lookup table. Tests verify correctness, not define the solution.
30 - Solutions should be robust, maintainable, and extendable.
31 - Follow best practices and sound software design principles.
32 - If a task is unreasonable, infeasible, or tests are incorrect, say so.
```

Step 3: Verify the file is loaded. Open the Kiro panel and check "Agent Steering & Skills". Because `inclusion: always` is set, this file loads automatically in every chat session across all workspaces.

Step 4: Test it by asking Kiro a question in any workspace. You should notice:

- Direct responses without flattery
- Verification from authoritative sources instead of relying on built-in knowledge
- No emoji in responses unless you ask for them

Global vs Workspace

Global steering files (`~/.kiro/steering/`) apply to all workspaces, while workspace steering files (`.kiro/steering/`) apply only to the current project. When both exist, they are merged. Use global for personal preferences and workspace for project-specific conventions.

Ref: [Steering Documentation](#)

1.4 Test Vibe Code

Workshop Exercise

Open Kiro IDE chat and paste the following prompt to test vibe coding.

Prompt to paste in Kiro Chat:

```
Research AWS documentation on Kiro Powers. Then create 5 pages presentation deck (PowerPoint-style with slide navigation) in html format named kiro-deck-vibe.html
```

This exercise demonstrates Kiro's ability to:

- Research documentation using web tools
- Generate a complete single-page application
- Follow complex multi-requirement prompts
- Produce working code with references

1.5 Install Workspace-scoped Steering Files

Steering files give Kiro persistent knowledge about your project. In this exercise, install a design system steering file and re-run the same prompt from 1.4 to see how it changes the output.

Step 1: Download the steering file:

macOS / LinuxWindows (PowerShell)

```
1 mkdir -p .kiro/steering
```

Save [design-system.md](#) into `.kiro/steering/`.

Step 2: Verify the file is loaded. Open the Kiro panel and check "Agent Steering & Skills". The file uses `inclusion: always`, so it is automatically loaded in every interaction.

Step 3: Open a new chat session and re-run the same prompt from 1.4:

```
Research AWS documentation on Kiro Powers. Then create 5 pages presentation deck (PowerPoint-style with slide navigation) in html format named kiro-deck-steering.html
```

Step 4: Compare the output with the result from 1.4. Notice how the steering file influences:

- Color palette: Brand Red (#EB2D2E), Dark Charcoal (#222222), consistent grays
- Typography: Open Sans font family, proper type scale
- Layout: 1080px container, section patterns, component styles
- Components: Button styles, navigation, hero section matching the design system

Key Insight

This demonstrates the power of steering: the same prompt produces different results when Kiro has design system context. No need to repeat conventions in every prompt.

Ref: [Steering Documentation](#)

1.6 Install Prebuilt Skills

Install all 3 sample skills from the [aws-samples/sample-strands-agents-agentskills](#) repo:

Skill	Description
file-processing	Process and analyze CSV, JSON, and text files with data transformation, cleaning, and visualization
pptx	Presentation creation, editing, and analysis for .pptx files
skill-creator	Guide for creating effective skills following best practices

Step 1: Clone the repo and copy all skills into your workspace:

macOS / LinuxWindows (PowerShell)

```
1 git clone https://github.com/aws-samples/sample-strands-agents-agentskills.git /tmp/agentskills
2 mkdir -p /tmp/skills
3 cp -r /tmp/agentskills/skills/file-processing .kiro/skills/
4 cp -r /tmp/agentskills/skills/pptx .kiro/skills/
5 cp -r /tmp/agentskills/skills/skill-creator .kiro/skills/
6 rm -rf /tmp/agentskills
```

Step 2: Verify all 3 skills appear in the Kiro panel under "Agent Steering & Skills".

Step 3: Test the `file-processing` skill. Download the sample data file:

[sample-sales-data.csv](#) (25 rows of sales data with regions, products, categories, and pricing)

Save it to your workspace, then ask Kiro to analyze it:

```
Analyze sample-sales-data.csv: give me summary statistics, revenue by region, top-selling products, and identify any trends over time. Show results as markdown tables. Use 111
```

(Optional) Follow up with:

```
from this information create presentation to report this analysis to executives
```

Step 4: Test the `skill-creator` skill:

```
Help me create a new skill for reviewing Ansible playbooks for security best practices.
```

Note

Skills activate automatically when your prompt matches the skill's description. If the skill does not load, retry using the slash command to invoke it directly (e.g., `/file-processing` or `/skill-creator`).

Ref: [AWS Sample Skills Repo](#) | [Agent Skills Directory](#)

1.7 Install Prebuilt Powers

Step 1: Open the Command Palette and run "Kiro: Configure Powers".

Step 2: Install these two powers:

- Build a Power:** Guided creation of custom powers with templates, best practices, and validation
- AWS Cost Optimization:** Analyze and optimize AWS resource costs

Step 3: Test **Build a Power**:

```
Help me build a new Kiro power for integrating with our Elasticsearch
```

Step 4: (Optional) Test **AWS Cost Optimization** by mentioning cost-related keywords:

```
Analyze my AWS infrastructure for cost optimization opportunities.
```

Check the context indicator to confirm the power activated.

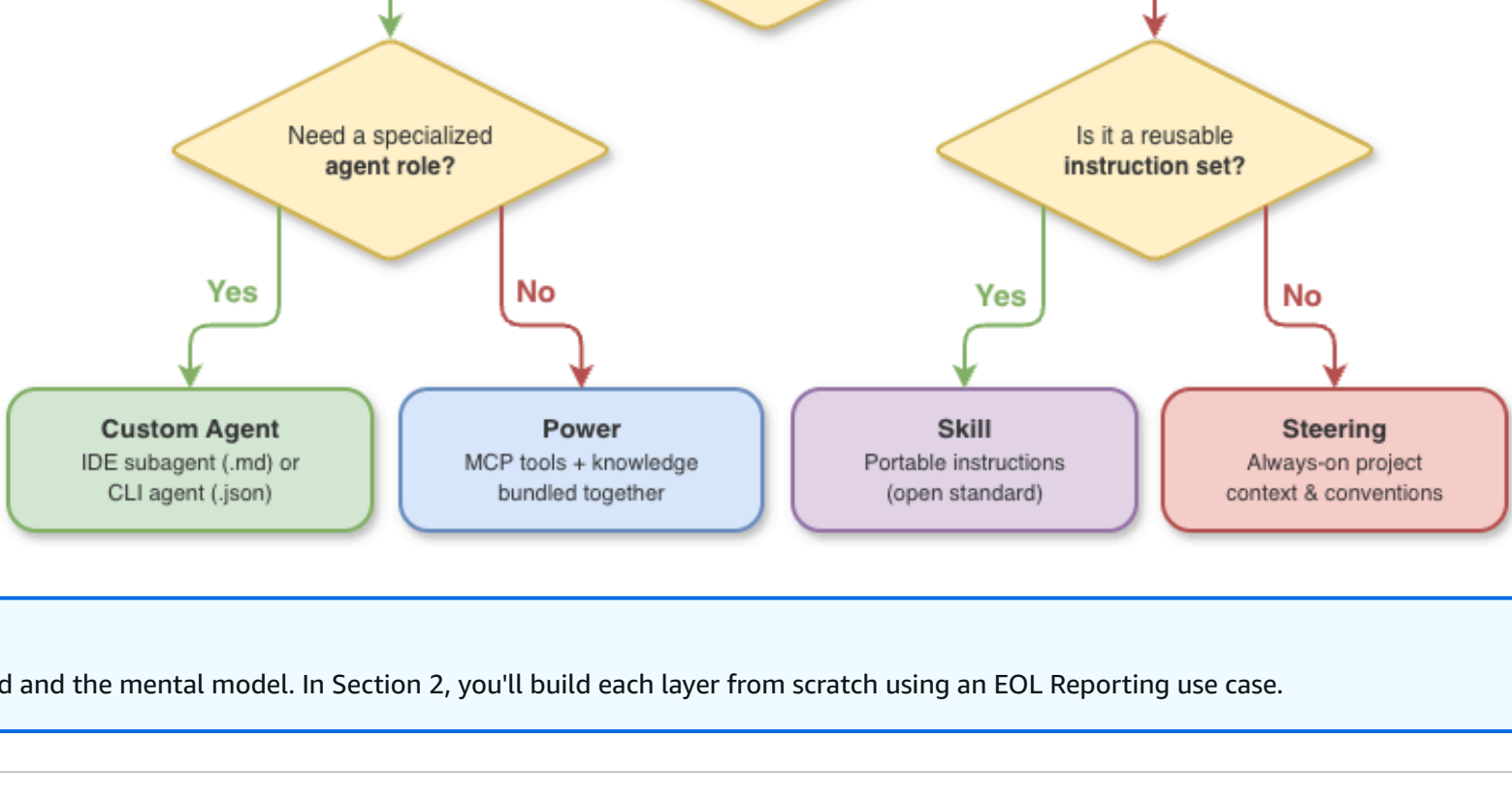
Ref: [Powers Marketplace](#) | [Powers Installation Guide](#)

1.8 Understanding Skills, Powers & Custom Agents

Feature Comparison

Feature	Steering	Skill	Power	Custom Agent
What is it?	Markdown files in <code>.kiro/steering/</code>	Portable instruction package (SKILL.md)	MCP tools + knowledge + workflows (SKILL.md)	IDE: Markdown + YAML (.md). CLI: JSON (.json)
Standard	Kiro-native	Open standard (agentskills.io)	Kiro-native (POWER.md + MCP)	Kiro-native (IDE and CLI formats differ)
Activation	<code>always</code> / <code>fileMatch</code> / <code>manual</code> / <code>auto</code>	On-demand via description match or /slash command	Automatic via keyword matching	IDE: <code>/name</code> , <code>auto-select</code> . CLI: <code>/agent swap</code>
Includes MCP?	No	No, instructions and scripts only	Yes, bundles MCP server config	IDE: <code>includeMcpjson</code> / <code>includePowers</code> . CLI: <code>mcpServers</code>
Best for	Project conventions, coding standards	Reusable workflows, review checklists	Tool integrations (Stripe, Supabase, Terraform)	Specialized roles (sre-agent, infra-reviewer)

Decision Tree: Which One to Use?



Ready to Build

You now have the tools installed and the mental model. In Section 2, you'll build each layer from scratch using an EOL Reporting use case.